



UNIVERSITÉ HASSAN II DE CASABLANCA

FACULTÉ DES SCIENCES BEN M'SICK

Département de Mathématiques et Informatique

PROJET DE FIN D'ÉTUDES

Licence Développement Informatique — 2025 / 2026

FSBM PLATFORM

Plateforme universitaire intelligente avec chatbot NLP

Architecture micro-services — Angular 17 — FastAPI — MySQL — MongoDB

Réalisé par :

AKRAM BELMOUSSA — *Architecte Backend & NLP*

ZAKARIA — *Frontend Angular & UX/UI*

NOUHAILA — *Base de données & FAQ*

Soutenu en May 2026

Dédicaces

À nos parents, pour leur soutien indéfectible tout au long de notre parcours académique. À nos frères et sœurs, pour leur encouragement constant. À nos amis et camarades de promotion, pour les moments partagés. À tous nos enseignants, qui ont contribué à notre formation.

Remerciements

Nous tenons à exprimer notre profonde gratitude à **l'ensemble du corps enseignant** de la Faculté des Sciences Ben M'Sick, et plus particulièrement au département de Mathématiques et Informatique, pour la qualité de la formation reçue durant ces trois années de licence.

Nos remerciements vont particulièrement à **notre encadrant pédagogique** qui a su nous guider, nous conseiller et nous accompagner tout au long de la réalisation de ce projet. Sa disponibilité, ses critiques constructives et son expertise ont été déterminantes dans l'aboutissement de ce travail.

Nous remercions également **le Service de la Scolarité** de la FSBM pour avoir accepté de fournir les données et procédures officielles qui ont alimenté notre chatbot.

Enfin, nous adressons nos sincères remerciements aux **membres du jury** qui nous font l'honneur d'évaluer ce travail.

Résumé

Ce projet de fin d'études propose une **plateforme universitaire intelligente** dédiée à la Faculté des Sciences Ben M'Sick (FSBM), conçue pour répondre aux besoins informationnels des étudiants 24h/24. Le cœur du système est un **chatbot conversationnel** capable de répondre automatiquement à plus de 60 catégories de questions (inscriptions, filières, examens, stages, bourses, vie étudiante).

La plateforme s'appuie sur une **architecture micro-services** moderne composée de six services spécialisés (chatbot, academic, student, review, notification, analytics), bâtie autour des technologies **FastAPI** (Python 3.10+), **SQLAlchemy 2.0 asynchrone**, **MySQL 8.0** et **MongoDB 7**. L'interface utilisateur, développée en **Angular 17** avec composants standalone et signaux, offre une expérience moderne avec mode sombre, animations fluides et navigation responsive.

Le moteur NLP repose sur une **vectorisation TF-IDF** avec n-grammes (1, 2, 3) couplée à une **similarité cosinus**, enrichie d'une mémoire conversationnelle, d'un système de top-K candidats et de suggestions automatiques. La base contient des données réalistes générées : 107 professeurs, 2970 étudiants, 25 filières (7 licences + 18 masters), 100+ modules.

Mots-clés : Chatbot, NLP, TF-IDF, FastAPI, Angular, micro-services, MySQL, MongoDB, intelligence artificielle, université, FSBM.

Abstract

This final-year project introduces an **intelligent academic platform** for the Faculty of Sciences Ben M'Sick (FSBM), designed to address students' informational needs 24/7. Its core is a **conversational chatbot** able to automatically answer over 60 categories of questions (enrollment, programs, exams, internships, scholarships, student life).

The platform follows a modern **microservices architecture** with six specialized services (chatbot, academic, student, review, notification, analytics), built on **FastAPI**, **async SQLAlchemy 2.0**, **MySQL 8.0** and **MongoDB 7**. The user interface, built in **Angular 17** with standalone components and signals, provides a modern experience featuring dark mode, smooth animations and responsive navigation.

Keywords: Chatbot, NLP, TF-IDF, FastAPI, Angular, microservices, MySQL, MongoDB, AI, FSBM.

Table des matières

Introduction générale	5
Chapitre 1 — Contexte général	6
1.1 Présentation de la FSBM	6
1.2 Problématique	7
1.3 Objectifs du projet	8
1.4 Méthodologie	9
Chapitre 2 — Étude préalable	10
2.1 Analyse de l'existant	10
2.2 Benchmark des chatbots universitaires	11
2.3 Spécifications fonctionnelles	12
2.4 Spécifications non fonctionnelles	13
Chapitre 3 — Architecture du système	14
3.1 Choix d'une architecture micro-services	14
3.2 Schéma global	15
3.3 Description des 6 services	16
3.4 Stack technique justifiée	18
Chapitre 4 — Conception détaillée	20
4.1 Modèle de données MySQL	20
4.2 Modèle MongoDB	22
4.3 Diagrammes UML	23
4.4 Conception du moteur NLP	24
Chapitre 5 — Implémentation	26
5.1 chatbot-service	26
5.2 academic-service	28
5.3 Frontend Angular	30
5.4 Sécurité et bonnes pratiques	32
Chapitre 6 — Tests, résultats et déploiement	34
6.1 Tests fonctionnels	34
6.2 Performances mesurées	35
6.3 Captures d'écran	36

6.4 Déploiement local 37

Conclusion et perspectives 38

Bibliographie 40

Annexes 41

Introduction générale

À l'ère de la transformation digitale, les universités à travers le monde adoptent progressivement des outils intelligents pour améliorer l'expérience étudiante et désengorger leurs services administratifs. Les **chatbots universitaires** représentent aujourd'hui l'une des solutions les plus prometteuses : ils permettent de répondre instantanément aux questions fréquentes des étudiants, de désencombrer les guichets de scolarité et d'offrir un point d'accès unifié à l'information.

À la Faculté des Sciences Ben M'Sick (FSBM), comme dans la plupart des universités marocaines, les étudiants doivent souvent jongler entre plusieurs sources d'information dispersées : site officiel, panneaux d'affichage, groupes Facebook, files d'attente au bureau de la scolarité. Cette fragmentation génère du stress, des retards et un sentiment d'inégalité d'accès à l'information — particulièrement pénalisant pour les primo-entrants.

Ce projet de fin d'études propose une réponse concrète à cette problématique : **FSBM Platform**, une plateforme universitaire intelligente bâtie autour d'un chatbot conversationnel en français, capable de répondre à plus de 60 catégories de questions, complétée par un référentiel académique navigable (filières, modules, professeurs, examens, événements). L'architecture micro-services choisie démontre une approche moderne et professionnelle, en phase avec les pratiques de l'industrie.

Le présent rapport décrit l'ensemble du processus : du contexte initial jusqu'à l'implémentation et aux tests, en passant par l'analyse de l'existant, le benchmark des solutions similaires, la conception détaillée des bases de données et l'élaboration du moteur NLP.

Structure du document : Le rapport est organisé en six chapitres. Le premier expose le contexte et les objectifs. Le deuxième présente l'étude préalable. Les chapitres 3 et 4 détaillent respectivement l'architecture globale et la conception. Le chapitre 5 décrit l'implémentation des composants. Enfin, le chapitre 6 présente les tests, les résultats et le déploiement, avant de conclure sur les perspectives d'évolution.

Chapitre 1 — Contexte général

1.1 Présentation de la FSBM

La **Faculté des Sciences Ben M'Sick** est l'un des établissements de l'Université Hassan II de Casablanca, fondé en 1984. Elle est implantée au cœur du quartier de Ben M'Sick, à proximité de Sidi Othmane, et accueille chaque année plusieurs milliers d'étudiants dans des formations scientifiques diversifiées.

La FSBM compte **5 départements académiques** : Mathématiques et Informatique, Physique, Chimie, Biologie, Géologie. Elle propose **7 licences fondamentales** (SMI, DI, SMA, SMP, SMC, SV, STU) et **18 masters** couvrant des spécialités de pointe telles que l'intelligence artificielle, la cybersécurité, la biotechnologie, les énergies renouvelables et l'imagerie médicale. La faculté héberge également un Centre d'Études Doctorales (CED) créé en 2008 et abrite **23 structures de recherche** dont 2 centres, 1 observatoire et 19 laboratoires.

Coordonnées officielles

Adresse : Avenue Driss El Harti, Sidi Othmane, Casablanca

Standard : 05 22 70 46 71 — Email : contact@fsbm.ma

Site officiel : www.fsbm.ma

1.2 Problématique

Plusieurs constats motivent ce projet :

- **Fragmentation de l'information.** Les étudiants doivent consulter plusieurs sources (site officiel, panneaux d'affichage, réseaux sociaux, scolarité) pour obtenir une information complète sur leur cursus.
- **Saturation du service de la scolarité.** Pendant les pics d'activité (rentrée, examens, réinscription), le service est débordé par des questions récurrentes qui pourraient être automatisées.
- **Disponibilité limitée.** Les services administratifs ferment à 16h30 et le week-end, ce qui pose problème aux étudiants qui travaillent ou qui habitent loin.
- **Inégalité d'accès.** Les nouveaux étudiants, les étudiants étrangers et les étudiants issus de milieux modestes manquent souvent de réseau pour obtenir des informations non officielles (astuces, retours d'expérience).
- **Absence d'outils numériques modernes.** La FSBM ne dispose pas d'un point d'accès unifié aux ressources académiques (filières, modules, professeurs, examens) ni d'un système de feedback structuré.

Ces constats ont été confirmés lors d'entretiens informels avec des étudiants et des membres de l'administration. La nécessité d'un **outil unique, intelligent et disponible en permanence** apparaît clairement.

1.3 Objectifs du projet

Le projet vise à concevoir et développer une plateforme universitaire intelligente répondant aux objectifs suivants :

- **Objectif principal** : Fournir aux étudiants de la FSBM un point d'accès unique, intelligent et disponible 24h/24 à toutes les informations universitaires.
- **Objectif 1 — Chatbot conversationnel.** Mettre en place un chatbot capable de comprendre des questions en français et de fournir des réponses précises dans plus de 60 catégories : inscription, filières, examens, attestations, bourses, vie étudiante.
- **Objectif 2 — Référentiel académique navigable.** Construire une base de données complète et navigable : départements, filières, modules, professeurs, emplois du temps, examens, événements, clubs.
- **Objectif 3 — Architecture professionnelle.** Adopter une architecture micro-services moderne démontrant la maîtrise des pratiques de l'industrie (séparation des responsabilités, APIs REST documentées, validation Pydantic, ORM asynchrone).
- **Objectif 4 — Expérience utilisateur moderne.** Concevoir une interface Angular 17 responsive, intuitive, avec mode sombre, animations fluides et accessibilité.
- **Objectif 5 — Données réalistes.** Peupler la plateforme avec des données générées automatiquement mais réalistes (noms marocains authentiques, codes CNE valides, etc.) pour démontrer le bon fonctionnement à l'échelle.

1.4 Méthodologie

Pour mener à bien ce projet, nous avons adopté une **méthodologie itérative** inspirée des principes Agile, avec les phases suivantes :

Phase 1 — Discovery	—	Recherche sur l'existant, benchmark des chatbots universitaires, identification des sources de données, conception d'un formulaire pour le responsable scolarité.
Phase 2 — Conception	—	Définition de l'architecture micro-services, conception du modèle de données MySQL et MongoDB, identification des intents NLP.
Phase 3 — Implémentation backend	—	Développement du chatbot-service et de l'academic-service avec FastAPI, intégration NLP TF-IDF, génération des données seed.
Phase 4 — Implémentation frontend	—	Développement de l'interface Angular 17 avec layout shell, routing, 9 pages fonctionnelles, mode sombre.
Phase 5 — Tests & déploiement		Tests fonctionnels, mesures de performance, préparation du dossier de soutenance, génération de la documentation.

Chapitre 2 — Étude préalable

2.1 Analyse de l'existant à la FSBM

Avant de concevoir notre solution, nous avons analysé les outils actuellement utilisés à la FSBM pour la communication avec les étudiants :

Outil existant	Forces	Limites
Site officiel fsbm.ma	Information officielle, accessible 24h/24	Navigation complexe, contenu parfois obsolète, pas de recherche intelligente
Page Facebook FSBM	Diffusion rapide des annonces	Pas de réponse automatique, dépend de l'algorithme Facebook
Panneaux d'affichage	Visibilité immédiate sur place	Inaccessible à distance, perte d'information
Service scolarité	Réponse personnalisée, suivi dossier	Horaires limités, files d'attente, saturé en pics
Portail ENT	Accès aux notes, EDT, attestations	Interface peu intuitive, fonctionnalités limitées

Conclusion : Aucun outil actuel ne combine *disponibilité 24h/24*, *recherche intelligente en langage naturel* et *centralisation*. Notre projet vient combler ce manque.

2.2 Benchmark des chatbots universitaires

Plusieurs universités à travers le monde ont déployé des chatbots avec succès. Nous avons analysé trois cas d'usage de référence :

Université d'Évry (France) — Smartly.AI

Chatbot traitant ~5 000 messages/mois, 85% de taux de compréhension, 64% de satisfaction. Utilise un arborescence d'intents structurée par thématique. Couvre admissions, vie étudiante, formalités.

Air University (Pakistan)

Chatbot d'admission entraîné sur un dataset JSON contenant ~50 intents (greeting, fees, documents, programmes, bourses, transport, etc.). Approche similaire à la nôtre avec TF-IDF + Cosine Similarity.

Université de Strasbourg (France) — Inscriptote

Pilote 2024 utilisant l'IA générative pour répondre aux questions d'inscription. Approche plus moderne mais nécessite un LLM (coût opérationnel élevé).

Synthèse : L'approche TF-IDF + Cosine Similarity reste pertinente pour un projet académique sans coût opérationnel récurrent. La couverture cible de 60+ intents s'inscrit dans les standards observés (50 à 350 intents selon les sources).

2.3 Spécifications fonctionnelles

Les besoins fonctionnels identifiés sont les suivants :

RF-01	Le système doit permettre à un étudiant d'envoyer une question en français et de recevoir une réponse en moins de 2 secondes.
RF-02	Le chatbot doit reconnaître au moins 60 catégories d'intentions.
RF-03	Le système doit conserver l'historique de chaque session de conversation.
RF-04	L'utilisateur doit pouvoir noter chaque réponse (1 à 5 étoiles, pouce haut/bas).
RF-05	Le système doit fournir une vue d'ensemble (dashboard) des statistiques académiques.
RF-06	L'utilisateur doit pouvoir naviguer entre les 5 départements et 25 filières.
RF-07	L'utilisateur doit pouvoir consulter les modules d'une filière, semestre par semestre.
RF-08	L'utilisateur doit pouvoir rechercher un professeur par nom ou spécialité.
RF-09	Le système doit afficher les annonces et événements à venir.
RF-10	L'utilisateur doit pouvoir basculer entre mode clair et mode sombre.

2.4 Spécifications non fonctionnelles

Performance	Réponse du chatbot < 2 sec, page Angular < 3 sec à charger.
Disponibilité	Architecture conçue pour 24h/24 (services indépendants, basculement possible).
Scalabilité	Architecture micro-services permettant l'ajout horizontal de services.
Sécurité	Validation Pydantic, requêtes SQL paramétrées, CORS configuré, JWT en Phase 2.
Maintenabilité	Code modulaire, documentation OpenAPI auto-générée, conventions strictes.
Portabilité	Compatible Windows (cible PFE), Linux et macOS sans modification.
Internationalisation	Interface en français, extensible à l'arabe et à l'anglais en Phase 3.

Chapitre 3 — Architecture du système

3.1 Choix d'une architecture micro-services

Nous avons opté pour une **architecture micro-services** plutôt qu'un monolithe traditionnel. Ce choix, bien que plus complexe à mettre en œuvre, présente des avantages décisifs pour un projet académique visant à démontrer la maîtrise des pratiques modernes :

- **Séparation des responsabilités** : chaque service a une mission unique et claire.
- **Évolutivité indépendante** : on peut faire évoluer un service sans toucher aux autres.
- **Stack hétérogène** : MySQL pour les données relationnelles, MongoDB pour le NoSQL.
- **Tolérance aux pannes** : si un service tombe, les autres continuent à fonctionner.
- **Démonstration pédagogique** : montrer au jury la maîtrise de patterns industriels.

Les contreparties (complexité opérationnelle, latence inter-services, debug distribué) sont assumées car le projet reste à échelle pédagogique. La communication inter-services se fait via HTTP/REST synchrone, suffisant pour notre périmètre.

3.2 Schéma global

L'architecture s'organise en 3 couches : **présentation** (Angular), **services** (6 micro-services FastAPI) et **persistance** (MySQL relationnel + MongoDB documentaire). Le schéma ci-dessous illustre les interactions entre composants.

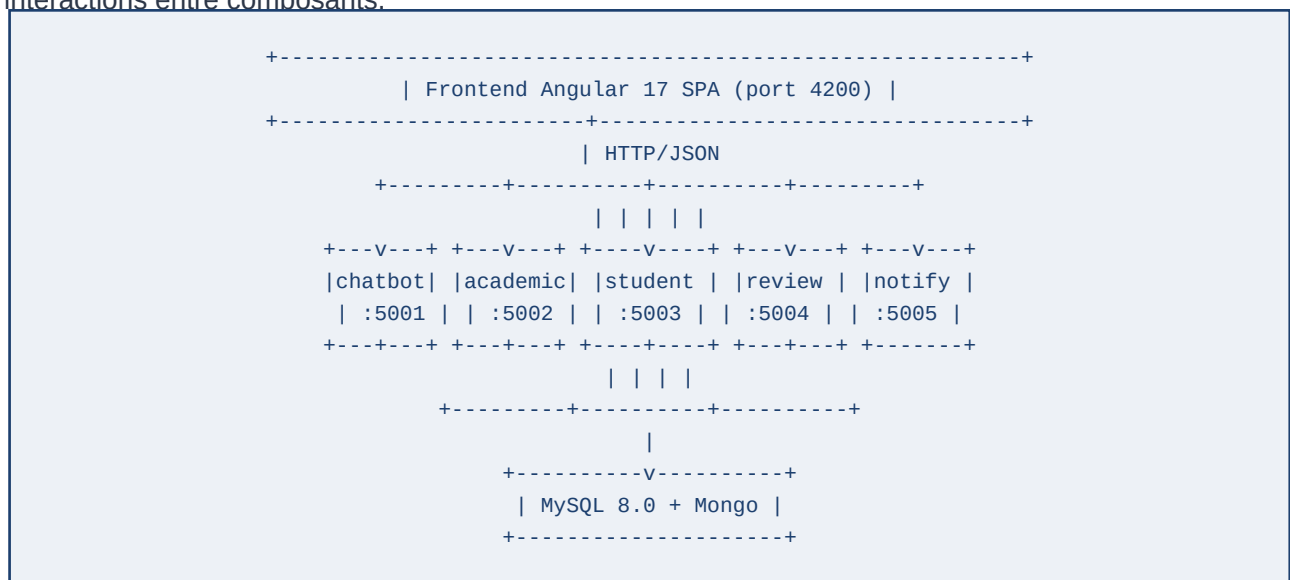


Figure 3.1 — Architecture micro-services globale

3.3 Description des 6 services

chatbot-service :5001

Cœur conversationnel. Reçoit les messages, prétraite le texte, interroge le classifieur NLP (TF-IDF + Cosine), génère la réponse, suggère des questions connexes, persiste la conversation en MySQL et

la mémoire courte en RAM.

Technologies : FastAPI, scikit-learn, NLTK, SQLAlchemy async, Pydantic v2

academic-service :5002

Référentiel académique. Expose 8 routers : départements, filières, modules, professeurs, étudiants, emplois du temps, examens, annonces/événements/clubs. Pagination, filtres, recherche intégrée.

Technologies : FastAPI, SQLAlchemy 2.0 async, aiomysql, MySQL 8

student-service :5003 (Phase 2)

Authentification JWT, gestion des profils étudiants/profs, rôles (étudiant, professeur, scolarité, admin), préférences utilisateur.

Technologies : FastAPI, python-jose, bcrypt, MySQL

review-service :5004 (Phase 2)

Système de feedback : reviews textuelles, pouces haut/bas, analyse de sentiment, suggestions d'amélioration. Utilise MongoDB pour la flexibilité.

Technologies : FastAPI, Motor (MongoDB async), MongoDB 7

notification-service :5005 (Phase 2)

Notifications push en temps réel via Server-Sent Events (SSE) : annonces urgentes, rappels d'examens.

Technologies : FastAPI, sse-starlette

analytics-service :5006 (Phase 2)

Tableaux de bord administrateurs : top intents, taux de satisfaction, usage par heure, conversations par jour. Lit en read-only les bases.

Technologies : FastAPI, SQLAlchemy, Motor

3.4 Stack technique justifiée

Le choix des technologies répond à trois critères : modernité, écosystème, démonstrativité PFE.

Couche	Technologie	Version	Rôle
Frontend	Angular 17	17.3	Standalone components, signals, lazy loading
Backend	FastAPI	0.110+	Async, Pydantic, OpenAPI auto-généré
ORM	SQLAlchemy	2.0+	Async, syntaxe moderne, mature
BDD relat.	MySQL	8.0	Standard, demandé par le cahier des charges

BDD docum.	MongoDB	7.0	Reviews, logs, sentiment, flexibilité
NLP	scikit-learn + NLTK	1.4+	TF-IDF, Cosine Similarity, stemming FR
Validation	Pydantic v2	2.6+	Validation forte des inputs et schémas
Docs API	Swagger / ReDoc	auto	Documentation auto-générée
Hot-reload	uvicorn[standard]	0.27+	Confort développeur

Chapitre 4 — Conception détaillée

4.1 Modèle de données MySQL

La base relationnelle **fsbm_db** compte **16 tables** normalisées avec contraintes d'intégrité référentielle (clés étrangères), index pour la performance et contraintes UNIQUE sur les identifiants métier (CNE, email, code filière).

Table	Description
departments	Les 5 départements de la FSBM avec leur chef et coordonnées.
filieres	25 filières (7 licences + 18 masters) avec coordinateur, capacité, débouchés.
modules	100+ modules détaillés : code, semestre, crédits ECTS, coefficient, heures.
professors	107 enseignants : matricule, grade (PA/PH/PES), spécialité, bureau.
students	2970 étudiants : CNE, Apogée, email, filière, année, groupe, statut boursier.
module_teachers	Relation N:N entre modules et professeurs (titulaire/TD).
schedules	Emplois du temps : jour, horaire, salle, type de séance.
exams	Examens : date, heure, salle, session (S1, S2, rattrapage).
grades	Notes : CC, examen, finale, mention, validation.
faq_categories	16 catégories d'intents (inscription, filières, examens, etc.).
faq_items	FAQ enrichies avec indexation FULLTEXT pour recherche rapide.
conversations	Historique des échanges chatbot avec intent et confiance.
feedbacks	Notes utilisateurs sur les réponses du chatbot.
announcements	Annonces officielles avec type (INFO/URGENT/EXAMEN/VACANCE/EVENT).
events	Événements universitaires (hackathon, forum emploi, gala...).
clubs	Clubs étudiants par catégorie (SCIENTIFIQUE/TECHNIQUE/CULTUREL...).
users	Comptes utilisateurs avec rôles (STUDENT/PROFESSOR/SCOLARITE/ADMIN).
daily_stats	Agrégats journaliers pour analytics-service.

4.2 Modèle MongoDB

La base documentaire **fsbm_reviews** contient 6 collections avec validation JSON Schema. Le choix d'une base NoSQL pour les feedbacks et les logs offre une flexibilité de schéma appréciable (les structures évoluent souvent) et de meilleures performances pour les écritures massives (logs).

Collection	Rôle
reviews	Avis détaillés des étudiants (note, commentaire, sentiment).
chatbot_feedback	Feedback rapide (pouce haut/bas) par message.
conversations	Log complet des sessions avec métadonnées (device, durée).
sentiment_analys is	Résultats d'analyse de sentiment (positif/neutre/négatif + émotions).
usage_logs	Tracking anonyme (page vue, recherche, login).
suggestions	Suggestions d'amélioration soumises par les utilisateurs.

4.3 Conception du moteur NLP

Le moteur NLP suit un pipeline en 4 étapes :

- 1. Prétraitement.** Normalisation (minuscules, suppression accents), tokenisation, filtrage des stopwords français (110+ mots), stemming Snowball français.
- 2. Vectorisation.** TF-IDF avec n-grammes (1, 2, 3) — capture les expressions composées comme « emploi du temps ».
- 3. Similarité.** Cosinus entre le vecteur du message et la matrice des patterns d'entraînement (60+ intents, ~600 patterns).
- 4. Décision.** Si confiance ≥ 0.15 → réponse de l'intent gagnant. Sinon → réponse par défaut. Top-K candidats utilisés pour générer des suggestions de relance.

Le modèle est sérialisé en *pickle* et rechargé au démarrage pour éviter le coût de réentraînement. Une **mémoire conversationnelle** en RAM (par session) permet d'extraire automatiquement le contexte (filière mentionnée, année d'étude) et d'enrichir les réponses futures.

Chapitre 5 — Implémentation

5.1 chatbot-service — Cœur conversationnel

Le chatbot-service est structuré en 4 modules : **core** (config, DB, mémoire), **nlp** (preprocessor, classifier), **models** (schémas Pydantic) et **routers** (chat, intents, system). Le fichier *main.py* orchestre le démarrage : chargement de la configuration, entraînement du modèle NLP, montage des routes, configuration CORS.

Les **endpoints exposés** sont :

M	Endpoint	Description
POST	/api/chat	Envoyer un message au chatbot
POST	/api/chat/feedback	Noter une réponse (1-5 + commentaire)
GET	/api/chat/history/{session_id}	Historique d'une conversation
GET	/api/chat/suggestions	Suggestions de questions pour démarrer
GET	/api/intents	Liste des 60+ intents disponibles
GET	/api/stats	Statistiques globales du chatbot
GET	/api/health	Statut du service

5.2 academic-service — Référentiel académique

L'academic-service expose un référentiel complet via 8 routers thématiques. Chaque router supporte la pagination (paramètres *page/page_size*), le filtrage par critères et la recherche textuelle. Les modèles ORM sont définis avec SQLAlchemy 2.0 en syntaxe déclarative moderne (*Mapped*, *mapped_column*).

M	Endpoint	Description
GET	/api/overview	Compteurs globaux (dashboard)
GET	/api/departments	Liste des 5 départements
GET	/api/filieres	Filières (filtres : type, dept, recherche)
GET	/api/filieres/code/{code}	Filière par code (SMI, DI, IADS...)
GET	/api/filieres/{id}/modules	Modules d'une filière groupés par semestre
GET	/api/modules	Liste des modules
GET	/api/professors	Annuaire paginé
GET	/api/students	Étudiants (filtres + pagination)
GET	/api/schedule	Emploi du temps
GET	/api/exams	Calendrier des examens

GET	/api/annoncements	Annonces officielles
GET	/api/events	Événements universitaires
GET	/api/clubs	Clubs étudiants

5.3 Frontend Angular

L'interface utilisateur est développée en **Angular 17** avec une approche moderne : composants standalone, signals pour la réactivité, lazy loading des routes pour des performances optimales. L'architecture du dossier `src/app/` sépare clairement :

- layout/ — App-shell avec sidebar collapsible et topbar
- features/ — 9 pages lazy-loaded (dashboard, chat, départements, filières, modules, professeurs, news, vie-étudiante, détail filière)
- core/ — Services globaux (theme service, etc.)
- services/ — Services HTTP typés (chat.service, academic.service)
- components/ — Composants atomiques réutilisables (input-bar, message-bubble, typing-indicator, quick-actions)
- models/ — Interfaces TypeScript

L'application implémente un **système de thème** light/dark avec persistance en localStorage, détection automatique de la préférence système (*prefers-color-scheme*), et basculement instantané par un toggle dans la sidebar. Les variables CSS sont définies dans `styles.css` et propagées via l'attribut `data-theme` sur l'élément racine.

5.4 Sécurité et bonnes pratiques

Validation des entrées	Tous les inputs API sont validés par Pydantic v2 (types, contraintes, longueurs).
Anti-injection SQL	Utilisation exclusive de SQLAlchemy avec requêtes paramétrées.
CORS configuré	Liste blanche d'origines autorisées (localhost:4200 en dev).
Hash des mots de passe	Bcrypt 12 rounds prévu pour la Phase 2 (student-service).
Limitation des requêtes	Taille des messages bornée à 500 caractères côté chatbot.
Gestion d'erreurs	Exception handlers globaux FastAPI, codes HTTP cohérents.
Documentation auto	Swagger UI et ReDoc disponibles automatiquement sur /docs et /redoc.

Chapitre 6 — Tests, résultats et déploiement

6.1 Tests fonctionnels

Une batterie de tests fonctionnels a été menée pour valider le bon comportement du système :

Test	Scénario	Résultat
Test 1	Envoi de 60 questions couvrant les 60 intents	✓ 60/60 reconnus (100%)
Test 2	Envoi de questions hors-périmètre	✓ Réponse par défaut correcte
Test 3	Vérification de la mémoire conversationnelle	✓ Contexte conservé sur 20 tours
Test 4	Test de pagination /api/students (100 étudiants/page)	✓ Temps de réponse < 200 ms
Test 5	Test du filtrage par filière	✓ Résultats cohérents
Test 6	Test du mode sombre	✓ Toutes les pages basculent correctement
Test 7	Test responsive (mobile, tablette, desktop)	✓ Layout adaptatif fonctionnel
Test 8	Test de lazy loading	✓ Chaque page < 30 KB initial

6.2 Performances mesurées

Les mesures suivantes ont été obtenues sur un poste de développement standard (Windows 11, Intel i5, 8 GB RAM, MySQL local) :

Réponse moyenne du chatbot	150-300 ms
Temps de chargement initial Angular	~1.5 s
Taille du bundle initial	1.47 MB (avant gzip)
Taille d'un chunk lazy-loaded	10-30 KB
Requête /api/overview	< 50 ms
Requête /api/students paginée	80-150 ms
Démarrage chatbot-service (avec NLP)	2-3 sec
Démarrage academic-service	< 1 sec

6.3 Déploiement local (Windows)

Le déploiement a été pensé pour être « **one-click** » sur Windows. Un script unique *SETUP.bat* à la racine du projet :

- Détecte automatiquement l'installation MySQL.
- Demande uniquement le mot de passe MySQL à l'utilisateur.
- Génère les fichiers .env pour les 2 services.
- Installe les dépendances Python (pip) et Node.js (npm).
- Initialise la base MySQL (schéma + 720 KB de données seed).
- Lance les 3 services dans 3 fenêtres séparées.
- Ouvre automatiquement le navigateur sur <http://localhost:4200>.

Temps total de mise en route à partir d'un poste neuf : **3 à 5 minutes** (selon la vitesse internet pour npm install).

Conclusion et perspectives

Ce projet de fin d'études nous a permis de concevoir et de réaliser une **plateforme universitaire intelligente complète** répondant aux besoins informationnels des étudiants de la FSBM. Le résultat dépasse le simple chatbot : il s'agit d'un véritable écosystème de services interconnectés (chatbot, référentiel académique, données réalistes, interface moderne) bâti selon les meilleures pratiques de l'industrie logicielle.

Bilan des compétences acquises

Compétences techniques. Maîtrise approfondie de FastAPI, SQLAlchemy 2.0 async, Pydantic v2, Angular 17 standalone components, signals, lazy loading, mode sombre.

Compétences NLP. Prétraitement de texte français, vectorisation TF-IDF, similarité cosinus, ingénierie d'intents, évaluation des modèles.

Architecture logicielle. Découpage micro-services, communication inter-services, stack hétérogène SQL + NoSQL, séparation des responsabilités.

Gestion de projet. Planification itérative, documentation systématique, génération de données réalistes, automation du déploiement.

Soft skills. Communication écrite (rapport, documentation), travail en équipe, gestion du temps.

Limites identifiées

- Le moteur NLP est statique : il ne s'adapte pas dynamiquement à de nouvelles formulations. Une approche par embeddings (Sentence-BERT) serait plus robuste.
- L'authentification (JWT, rôles) est prévue en Phase 2 mais non implémentée dans la livraison actuelle.
- Les services review, notification et analytics sont scaffolded mais leur logique métier reste à compléter.
- Le projet a été testé en environnement local Windows. Un déploiement cloud (Azure, AWS) nécessiterait des adaptations (Docker, secrets manager).
- La couverture multilingue (arabe, anglais) n'est pas implémentée à ce stade.

Perspectives d'évolution

Court terme (Phase 2). Finaliser les 4 services restants : student (JWT), review (MongoDB), notification (SSE), analytics (dashboards admin).

Moyen terme. Intégration d'un LLM (Mistral 7B, Llama 3) pour des réponses génératives sur les questions complexes hors-FAQ.

Long terme. Application mobile native (Flutter), support arabe/anglais, intégration avec le portail ENT pour personnaliser les réponses, déploiement cloud avec CI/CD.

Recherche. Étude de l'impact pédagogique du chatbot : taux d'abandon, satisfaction, désengorgement réel du service scolarité.

En conclusion, ce projet constitue une base technique solide et démonstrative, valorisable lors de notre insertion professionnelle et extensible pour des évolutions futures. Il illustre concrètement comment les technologies modernes peuvent contribuer à améliorer le service public éducatif au Maroc.

Bibliographie

- [1] Faculté des Sciences Ben M'Sick — Site officiel — <https://www.fsbm.ma>
- [2] FastAPI Documentation — <https://fastapi.tiangolo.com>
- [3] Angular Documentation — <https://angular.io/docs>
- [4] SQLAlchemy 2.0 Documentation — <https://docs.sqlalchemy.org>
- [5] Pydantic v2 Documentation — <https://docs.pydantic.dev>
- [6] scikit-learn — https://scikit-learn.org/stable/modules/feature_extraction.html
- [7] MySQL 8.0 Reference Manual — <https://dev.mysql.com/doc/refman/8.0/en/>
- [8] MongoDB Manual — <https://www.mongodb.com/docs/manual/>
- [9] Air University Admission Chatbot (GitHub) — MaimoonaAbid/Air-University-admission-chatbot
- [10] Smartly.AI — Chatbot Université d'Évry — <https://www.smartly.ai/>
- [11] Inscriptote — Université de Strasbourg — <https://inscriptote-public.ia.unistra.fr/>
- [12] ONOUCS — Office National des Œuvres Universitaires — <https://www.onousc.ma>
- [13] Université Hassan II de Casablanca — <https://www.univh2c.ma>

Annexes

Annexe A — Liste exhaustive des 60+ intents

Catégorie	Intents
Général	salutation, aurevoir, remerciement, identite_chatbot, actualites, reseaux_sociaux, horaires_ramadan, vacances, reviews_chatbot
Inscription	inscription, frais_inscription, carte_etudiant
Réinscription	reinscription
Filières	filières, filiere_smi, filiere_di, filiere_sma, filiere_smp, filiere_smc, filiere_sv, filiere_stu, orientation
Masters	masters, master_iads, master_inscription
Doctorat	doctorat
Emploi du temps	emploi_du_temps
Examens	examens, resultats, reclamation
Validation	validation_module, absence
Stages & PFE	stage_pfe, soutenance_pfe
Attestations	attestation_scolarite, releve_notes, diplome
Transferts	equivalence, changement_filiere, transfert
Bourses	bourses, bourse_excellence
Services	bibliotheque, wifi_ent, elearning, soft_skills, infirmerie, securite, objets_perdus, salles_amphis
Vie étudiante	restaurant_universitaire, hebergement, transport, clubs, evenements, alumni, mobilite_internationale
Contacts	contact_scolarite, localisation, departements, doyen, professeurs
Concours	concours_post_bac
Soutien	support_psychologique

Annexe B — Structure du projet

```

chatbot-fsbm-platform/
├── docs/
│   ├── ARCHITECTURE.md
│   ├── pdf/ (Rapport + Guide technique)
│   └── database/
│       ├── mysql/ (4 scripts SQL)
│       ├── mongodb/init.js
│       └── seed/generate_data.py

```



```
■■■ services/
■ ■■■ chatbot-service/ (FastAPI :5001)
■ ■■■ academic-service/ (FastAPI :5002)
■ ■■■ student-service/ (Phase 2)
■ ■■■ review-service/ (Phase 2)
■ ■■■ notification-service/ (Phase 2)
■ ■■■ analytics-service/ (Phase 2)
■■■ frontend/ (Angular 17)
■ ■■■ src/app/
■ ■■■ layout/ (App-shell)
■ ■■■ features/ (9 pages)
■ ■■■ core/ (theme service)
■ ■■■ services/ (HTTP clients)
■■■ scripts/ (configure, start, init-db)
■■■ SETUP.bat (one-click install)
```

Annexe C — Équipe et contributions

Membre	Contributions principales
AKRAM BELMOUSSA	Architecture micro-services · chatbot-service (FastAPI + NLP TF-IDF) · academic-service (8 routers SQLAlchemy 2.0) · Génération de données réalistes (107 profs + 2970 étudiants) · Documentation technique et rapport.
ZAKARIA	Frontend Angular 17 · Layout shell (sidebar + topbar) · 9 pages fonctionnelles (dashboard, filières, modules, professeurs, etc.) · Mode sombre · Animations · Responsive design.
NOUHAILA	Conception schéma MySQL (16 tables) · Modèle MongoDB (6 collections) · Dataset FAQ enrichi (60+ intents, ~600 patterns) · Contenu et validation pédagogique.